Evaluating Emerging Supercomputers via the MFC Flow Solver

Benjamin Wilfong[1], Anand Radhakrishnan[1], Henry Le Berre[1], Tanush Prathi[1], Stephen Abbott[2], Spencer H. Bryngelson[1]

HPCTESTS at SC ‘25

November 21st, 2025

1: Georgia Institute of Technology
2: Hewlett Packard Enterprise (HPE)





CPC
Computational Physics Group

What does MFC offer?

- Compressible fluids solver
- System of PDEs
- Proven performance record
- Growing user base
- Good tool for test suites

2025 ACM Gordon Bell Prize Finalist



Shock bubble interaction

What does MFC stress on systems?

$$\frac{\partial \alpha_i \rho_i}{\partial t} + \nabla \cdot (\alpha_i \rho_i \boldsymbol{u}) = 0$$

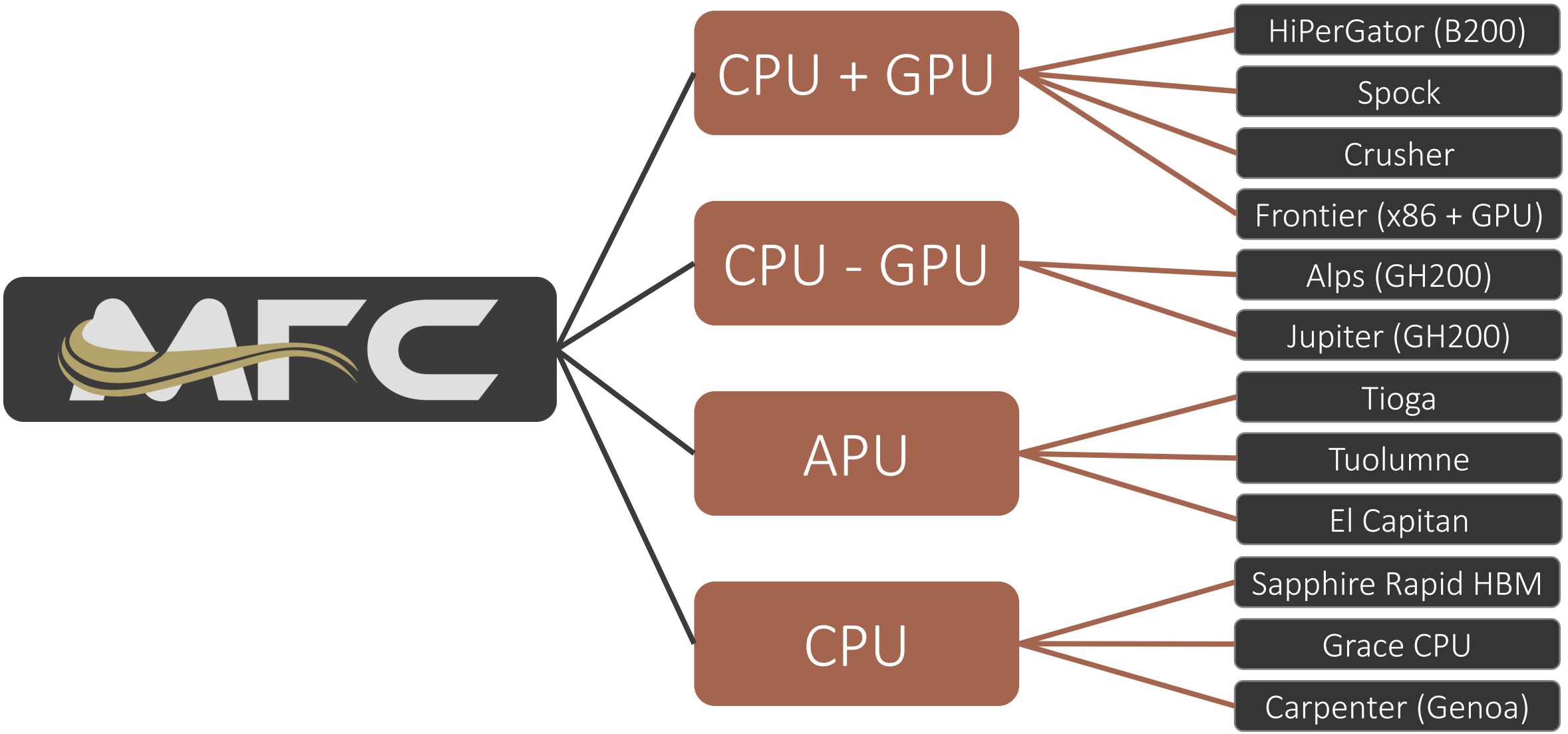
$$\frac{\partial \rho \boldsymbol{u}}{\partial t} + \nabla \cdot (\rho \boldsymbol{u} \otimes \boldsymbol{u} + p\boldsymbol{I} - \boldsymbol{T}^v) = 0$$

$$\frac{\partial (\rho E)}{\partial t} + \nabla \cdot ((\rho E + P)\boldsymbol{u} - \boldsymbol{T}^v \cdot \boldsymbol{u}) = 0$$

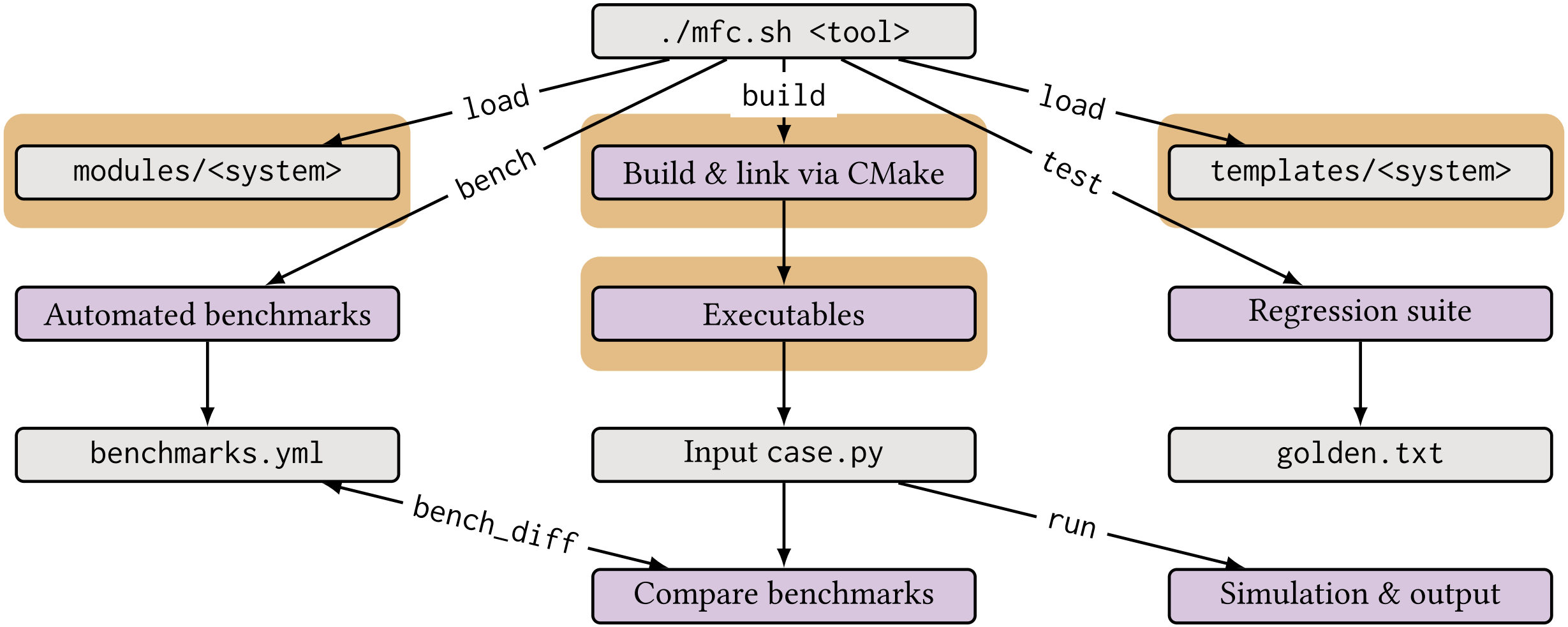
$$\frac{\partial \alpha_1}{\partial t} + \boldsymbol{u} \cdot \nabla \alpha_1 = K \nabla \cdot \boldsymbol{u}$$

- 3D + Time → expensive
- Rectilinear grid distributes to thousands of processes
- No global communication
- Large parallel I/O requirements (~100s of TB)
- Memory-bound stencil computations
- Unified memory perf.

What hardware does MFC use?



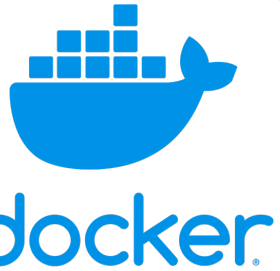
Getting started with MFC



Getting the code – Lots of options!



Homebrew for MacOS





Docker and Singularity for commodity hardware



Build from source

Module setup

System name and identifier

Modules used for all builds

Modules used for CPU builds

Modules used for gpu builds

Build time environment vars.

Example module file entry

```
d       NCSA Delta
d-all python/3.11.6
d-cpu gcc/11.4.0 openmpi
d-gpu nvhpc/24.1 cuda/12.3.0 openmpi/4.1.5+cuda cmake
d-gpu CC=nvc CXX=nvc++ FC=nvfortran
d-gpu MFC_CUDA_CC=80,86
```

```
source ./mfc.sh load -c d -m g
```

Templating - Overview

- One-time template setup via mako
- Enables
  - Adherence to scheduler differences
  - Module loading
  - Run-time environment variables
  - System dependent bindings/affinity
  - Interactive and batch run commands
  - Automatic profiling commands

```
#!/usr/bin/env bash
% if engine == 'batch':
    #SBATCH --nodes=${nodes}
    #SBATCH --ntasks-per-node=${tasks_per_node}
    #SBATCH --job-name="${name}"
    #SBATCH --output="${name}.out"
    #SBATCH --time=${walltime}
    #SBATCH --cpus-per-task=7
    #SBATCH -C nvme
    % if gpu:
        #SBATCH --gpus-per-task=1
        #SBATCH --gpu-bind=closest
    % endif
    % if account:
        #SBATCH --account=${account}
    % endif
    % if partition:
        #SBATCH --partition=${partition}
    % endif
    % if quality_of_service:
        #SBATCH --qos=${quality_of_service}
    % endif
    % if email:
        #SBATCH --mail-user=${email}
        #SBATCH --mail-type="BEGIN, END, FAIL"
    % endif
% endif

ok ":) Loading modules:\n"
cd "${MFC_ROOT_DIR}"
% if engine == 'batch':
. ./mfc.sh load -c f -m ${'g' if gpu else 'c'}
% endif
cd - > /dev/null
echo

% if gpu:
    export MPICH_GPU_SUPPORT_ENABLED=1
% else:
    export MPICH_GPU_SUPPORT_ENABLED=0
% endif

%if unified:
    export CRAY_ACC_USE_UNIFIED_MEM=1
% endif

ulimit -s unlimited

% for target in targets:
    % if engine == 'batch':
        # Broadcast binary to compute nodes
        sbcast --send-libs -pf ${target.get_install_binpath(case)} /mnt/bb/$USER/${target.name}
    % endif

    (set -x; srun --unbuffered \
    % if engine == 'interactive':
            --unbuffered --nodes ${nodes} --ntasks-per-node ${tasks_per_node} \
            --cpus-per-task 7 \
        % if gpu:
            --gpus-per-task 1 --gpu-bind closest \
        % endif
        ${profiler} "${target.get_install_binpath(case)}")
    % else:
        ${profiler} "/mnt/bb/$USER/${target.name}")
    % endif
% endfor
```

Templating – Run time variables

```
% if engine == 'batch':
    #SBATCH --nodes=${nodes}
    #SBATCH --ntasks-per-node=${tasks_per_node}
    #SBATCH --job-name="${name}"
    #SBATCH --output="${name}.out"
    #SBATCH --time=${walltime}
    #SBATCH --cpus-per-task=7
    #SBATCH -C nvme
    % if gpu:
        #SBATCH --gpus-per-task=1
        #SBATCH --gpu-bind=closest
    % endif
    % if account:
        #SBATCH --account=${account}
    % endif
    % if partition:
        #SBATCH --partition=${partition}
    % endif
    % if quality_of_service:
        #SBATCH --qos=${quality_of_service}
    % endif
    % if email:
        #SBATCH --mail-user=${email}
        #SBATCH --mail-type="BEGIN, END, FAIL"
    % endif
% endif
```

Loading modules

```
ok ":) Loading modules:\n"
cd "${MFC_ROOT_DIR}"
% if engine == 'batch':
. ./mfc.sh load -c f -m ${'g' if gpu else 'c'}
% endif
cd - > /dev/null
```

Run time variables

```
% if gpu:
    export MPICH_GPU_SUPPORT_ENABLED=1
% else:
    export MPICH_GPU_SUPPORT_ENABLED=0
% endif

% if unified:
    export CRAY_ACC_USE_UNIFIED_MEM=1
% endif

ulimit -s unlimited
```

Templating – Executables

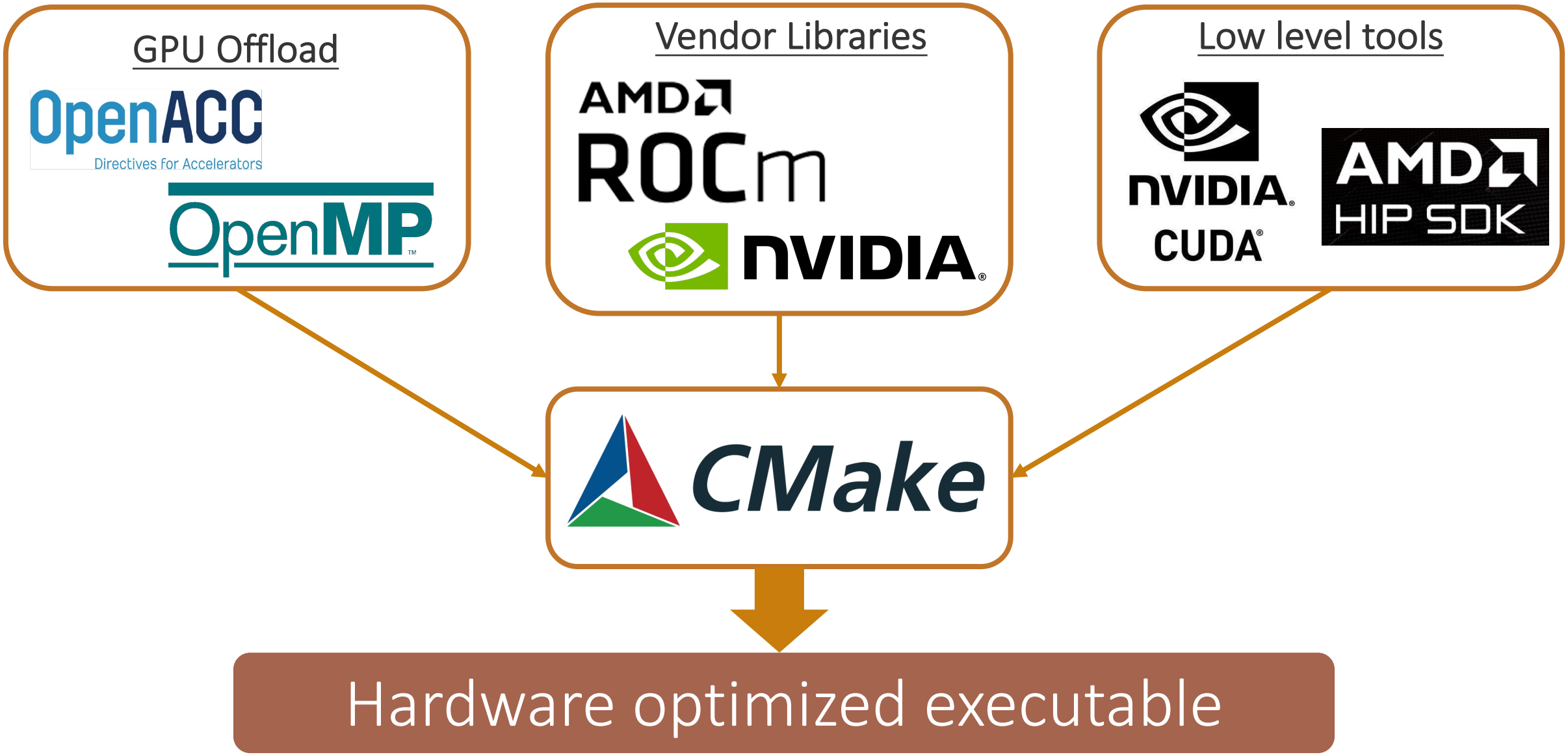
```
1 % for target in targets:
2     % if engine == 'batch':
3         # Broadcast binary to compute nodes
4         sbcast --send-libs -pf ${target.get_install_binpath(case)} \
5                 /mnt/bb/$USER/${target.name}
6     % endif
7
8     (set -x; srun \
9     % if engine == 'interactive':
10            --unbuffered --nodes ${nodes} --ntasks-per-node ${tasks_per_node} \
11            --cpus-per-task 7 \
12        % if gpu:
13            --gpus-per-task 1 --gpu-bind closest \
14        % endif
15        ${profiler} "${target.get_install_binpath(case)}")
16    % else:
17        ${profiler} "/mnt/bb/$USER/${target.name}")
18    % endif
19 % endfor
```

Broadcast executables for batch jobs

Run with interactive specific options and auto generated profiler command

Run with broadcast binary and auto generated profiler command in batch mode

Build from source for full control



GPU offloading capability

Compiler	OpenMP			OpenACC		
	NV GPUs	AMD GPUs	FP16 Atomics	NV GPUs	AMD GPUs	FP16 Atomics
AMD	✗	✔	✔	✗	✗	✗
CCE	✔	✔	✗	✔	✔	✗
NVHPC	✔	✗	?	✔	✗	✔

- Two/three vendors making supercomputers → Portability!
- OpenMP better supported, OpenACC faster
- Need to support everything for portable performance
- New directives for novel hardware?
  - NextSilicon Maverick-2 already supported by OpenMP!

Portable GPU offload using macros

```
$:GPU_PARALLEL_LOOP(collapse=3, private='[...]')
do l = 0, p       ! Z-direction
  do k = 0, n     ! Y-direction
    do j = 0, m   ! X-direction
        ! Core kernel
        ! O(100) - O(1000) operations
    end do
  end do
end do
$:END_GPU_PARALLEL_LOOP
```

FYPP

```
!$acc parallel loop vector gang collapse(3) &
!$acc default(present) private(...)
do l = 0, p       ! Z-direction
  do k = 0, n     ! Y-direction
    do j = 0, m   ! X-direction
        ! Core kernel
        ! O(100) - O(1000) operations
    end do
  end do
end do
!$acc end parallel loop
```

OpenACC

```
!$omp target teams distribute parallel do simd &
!$omp defaultmap(firstprivate:scalar) &
!$omp defaultmap(tofrom:aggregate) &
!$omp defaultmap(present:allocatable) &
!$omp defaultmap(present:pointer) &
!$omp collapse(3) private(...)
do l = 0, p       ! Z-direction
  do k = 0, n     ! Y-direction
    do j = 0, m   ! X-direction
        ! Core kernel
        ! O(100) - O(1000) operations
    end do
  end do
end do
!$omp end target teams distribute parallel do
```

OpenMP

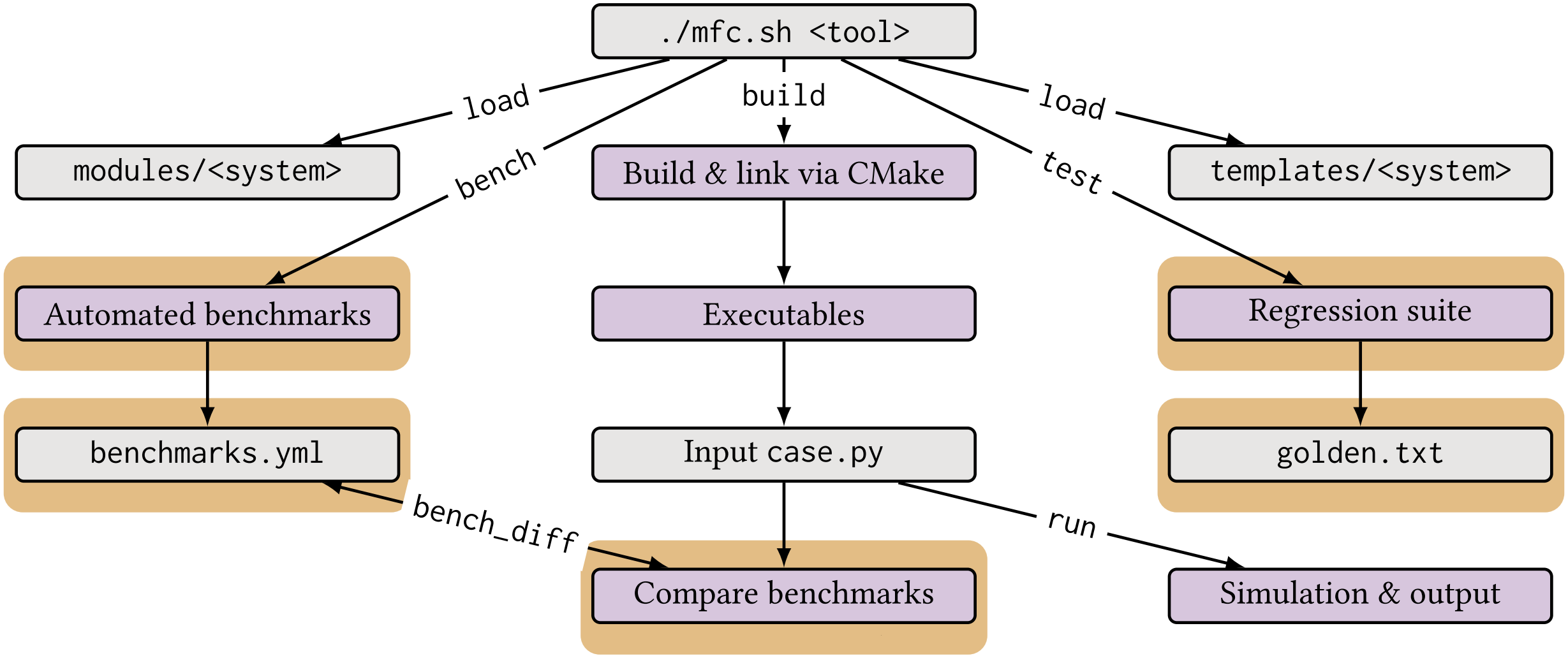
Developer writes *one* version of the code, with directives based on OpenACC

FYPP reads the source and passes it to FYPP

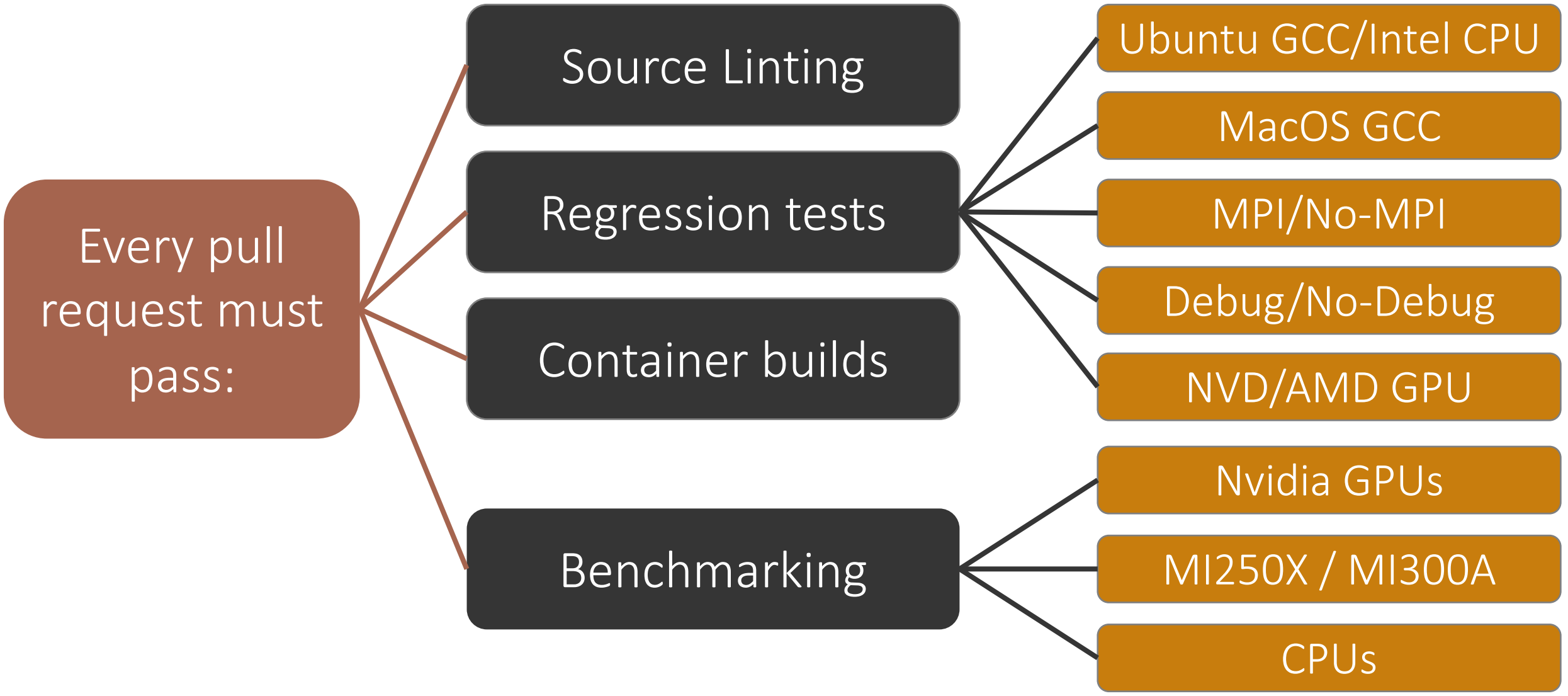
FYPP creates OpenMP directives from the OpenACC ones

The directives are replaced with macros used in the code

Continuous integration



Continuous Integration



Regression testing

Easy to add tests for new features

Add options for all cases

Loop over fine-grain options

```
def alter_igr():
    stack.push('IGR',{'igr': 'T',  'alf_factor': 10, 'num_igr_iters': 10,
                      'elliptic_smoothing': 'T', 'elliptic_smoothing_iters': 10,
                      'num_igr_warm_start_iters': 10})

    for order in [3, 5]:
        stack.push(f"igr_order={order}", {'igr_order': order})

        cases.append(define_case_d(stack, 'Jacobi', {'igr_iter_solver': 1}))
        if order == 5:
            cases.append(define_case_d(stack, 'Gauss Seidel', {'igr_iter_solver': 2}))

        stack.pop()

    stack.pop()
```

- Results compared between golden files stored in git
- Metadata files provide details on creation
- Ability to add new variables without rewriting existing results (> integrity)

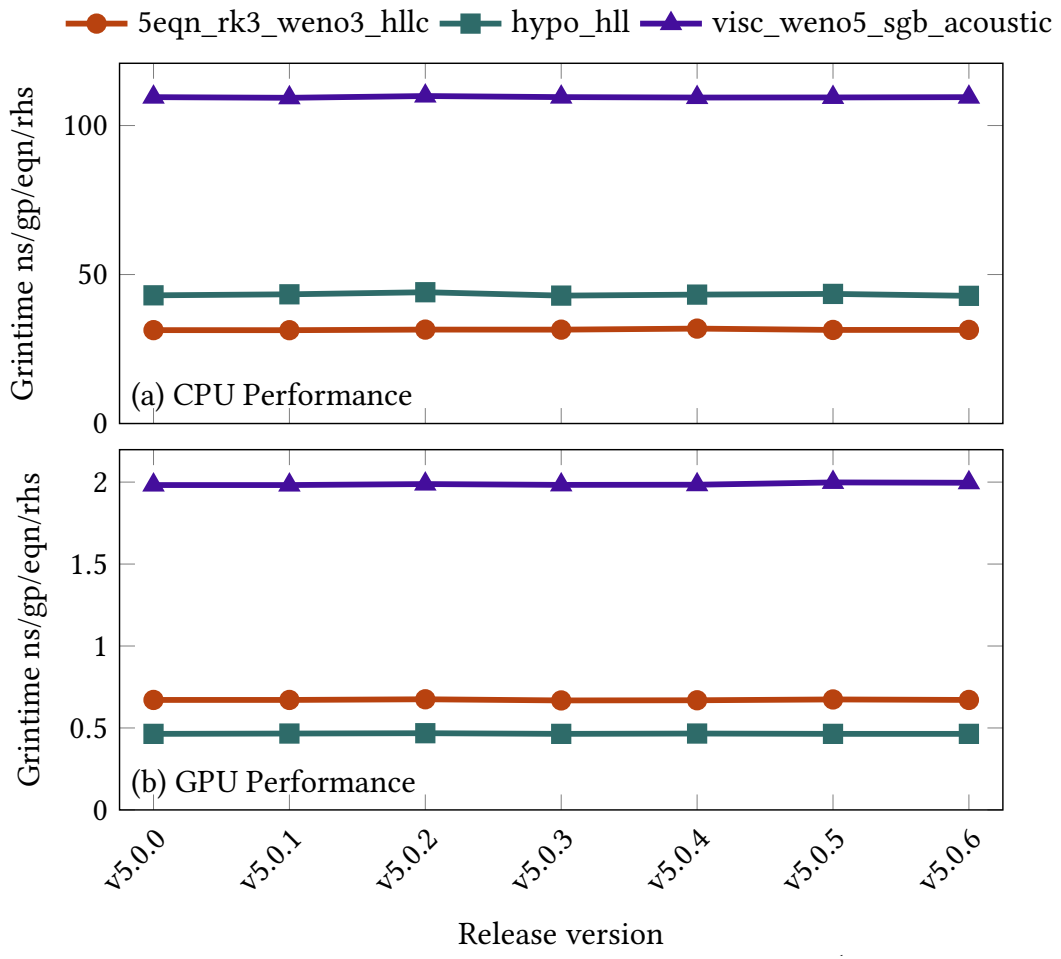
Automatic benchmarking

Figure of merit:

nanoseconds/grid cell/equation/rhs

Every pull request compared to main branch

Upcoming feature: benchmark performance over time plotted in documentation



Bottlenecks and errors identified

Private arrays with runtime sizes

Refactoring into routines that fail to inline

Incorrect workgroup size in early CCE compilers

Atomics poorly supported in lower precisions (FP16)

Demonstrated performance

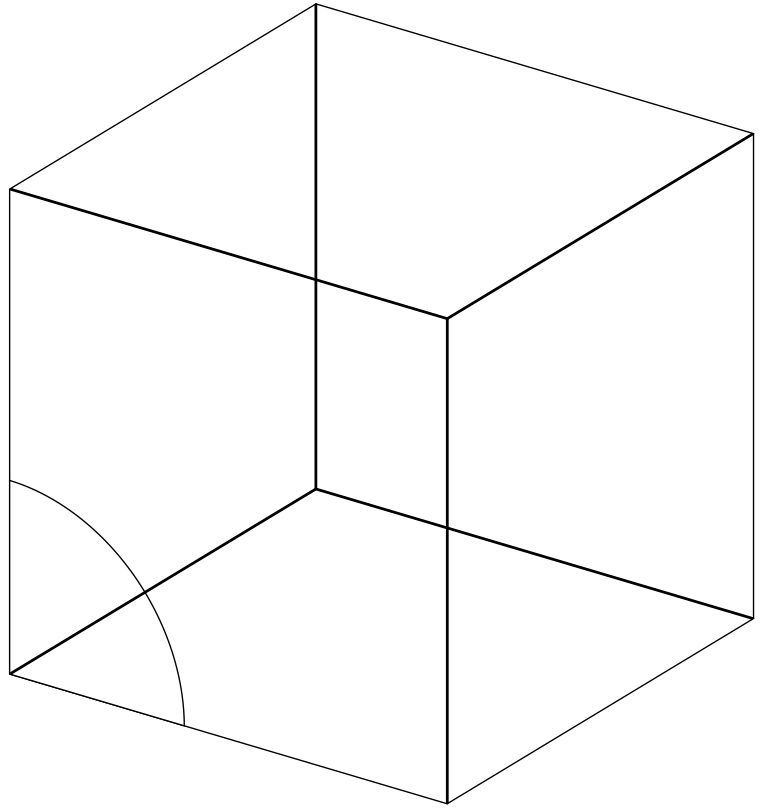
Standardized performance case

Common two–fluid problem

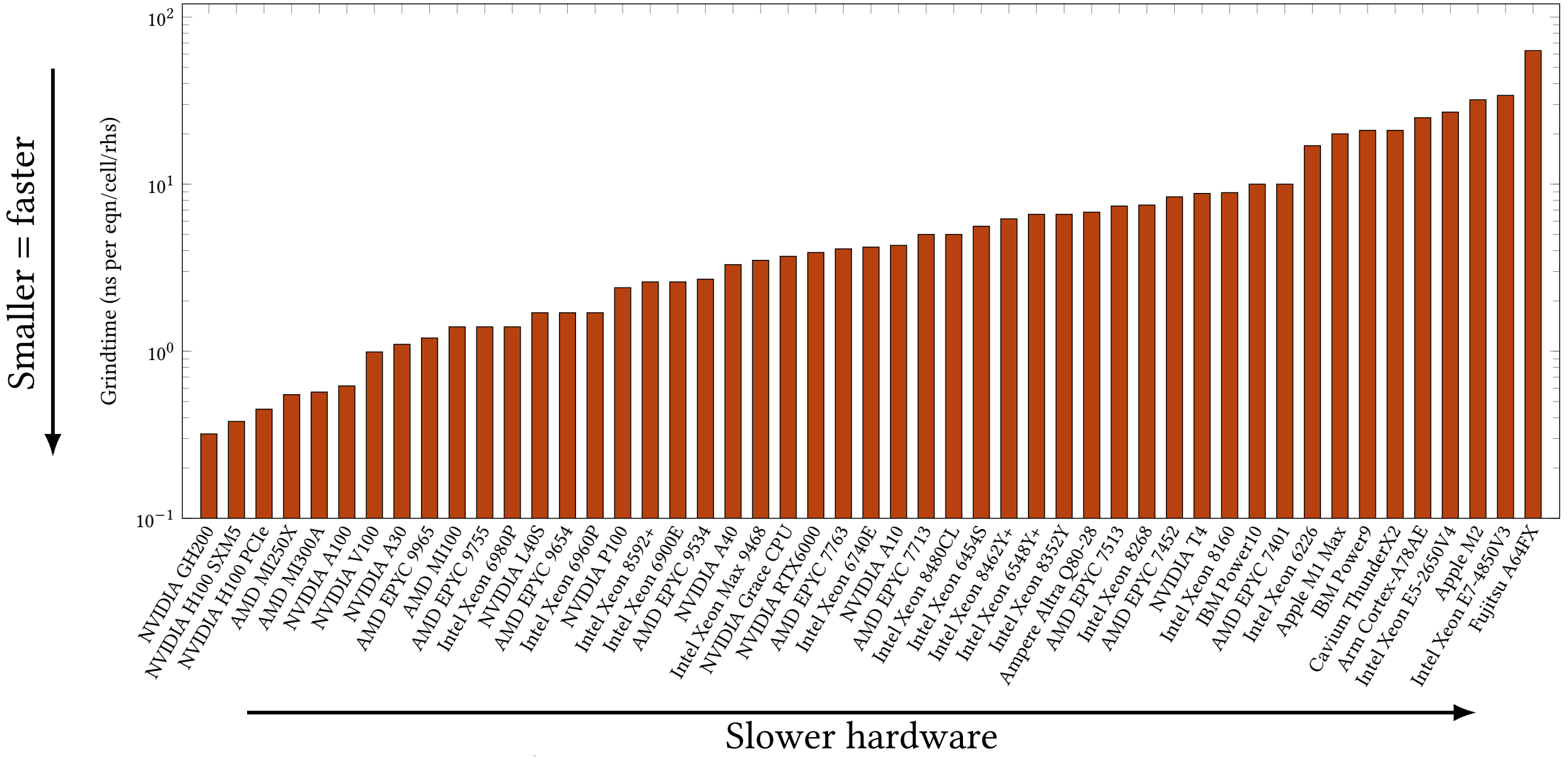
Standardized case file and problem size

Compares CPUs to GPUs with normalized QOI

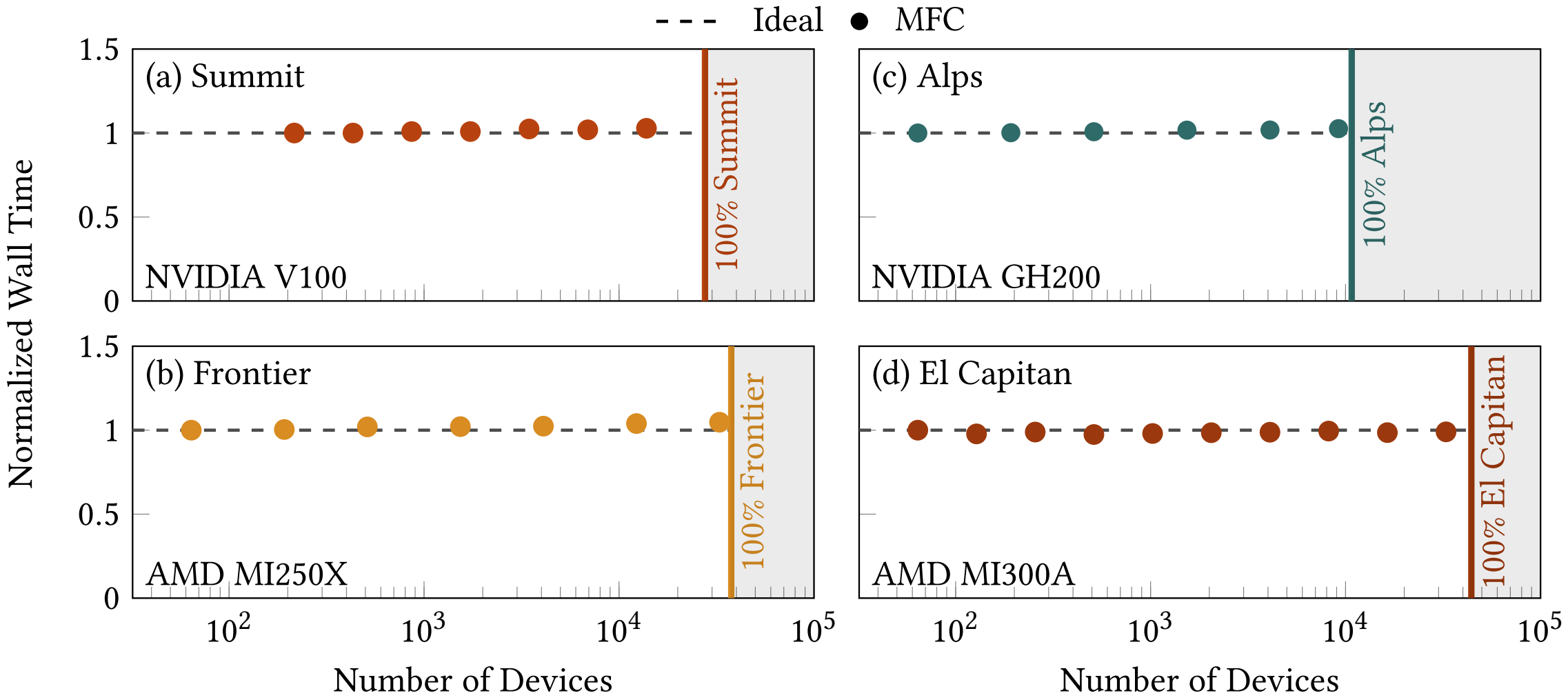
Documented results on ~50 architectures



Standardized performance case results

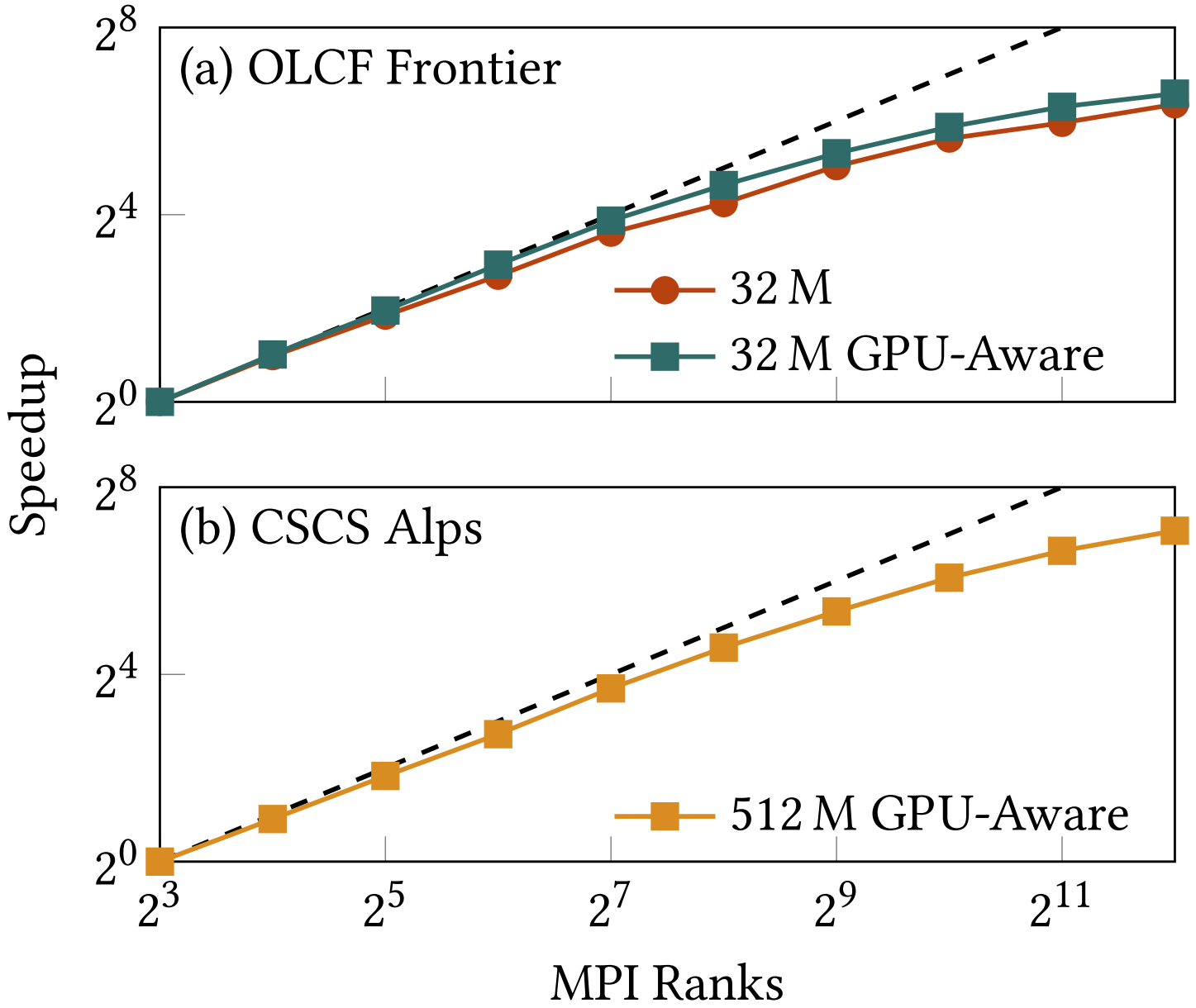


Weak scaling performance



Within 5% of ideal over 100-1000x increase in device count

Strong scaling performance



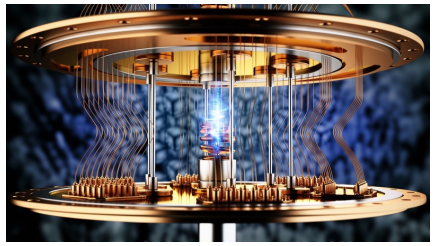
Impressive strong scaling performance over large increase in device count

Reminder: No global or collective operations

Limitations









Lacks support for exotic new hardware

When directives and systems come, we'll use them

Conclusion



- Easy to use
- Portable
- Compressible fluids solver
- Proven performance on leadership systems
- Growing user base

A good tool for use with your systems test suite

Thank you!

mflowcode.github.io



Acknowledgments:

Lawrence Livermore National Laboratory

National Nuclear Security Administration

U.S. DEPARTMENT OF ENERGY

OAK RIDGE National Laboratory | LEADERSHIP COMPUTING FACILITY

NSF

ACCESS